

Unidad 3 – Request Body y Validación Avanzada en FastAPI

1 Fundamentos: HTTP y el Ciclo Request-Response

El protocolo HTTP funciona mediante un ciclo:

1. Cliente envía una **request**
2. Servidor procesa
3. Servidor devuelve una **response**

Una request contiene:

- Método (GET, POST, PUT, PATCH, DELETE)
- Headers
- Body (opcional)

Una response contiene:

- Status code
- Headers
- Response body

◆ Familias de códigos HTTP

200–299 → Éxito

- 200 OK
- 201 Created

400–499 → Error del cliente

- 404 Not Found
- 422 Unprocessable Entity

500–599 → Error del servidor

2 ¿Qué es el Request Body?

El **request body** es el cuerpo de la petición HTTP donde el cliente envía datos al servidor.

En APIs modernas:

👉 Generalmente viaja en formato JSON.

Se usa principalmente con:

- POST → crear recurso
- PUT → actualizar completamente
- PATCH → actualización parcial

No es recomendado en GET o DELETE.

3 Pydantic y BaseModel

FastAPI utiliza Pydantic para:

- Validar datos
- Convertir tipos
- Generar documentación
- Serializar respuestas

◆ Modelo Pydantic

Un modelo es una clase que hereda de BaseModel.

```
from pydantic import BaseModel
```

```
class Item(BaseModel):  
    name: str  
    price: float  
    description: str | None = None  
    tax: float | None = None
```

¿Qué hace Pydantic?

Cuando llega un JSON:

1. Lo convierte a objeto tipado
2. Valida campos obligatorios
3. Valida tipos
4. Aplica valores por defecto
5. Genera esquema OpenAPI

Si algo falla → FastAPI responde automáticamente 422.

4 Ejemplo Básico – POST con Request Body

```
@app.post("/items/")  
async def create_item(item: Item):  
    return item
```

🔗 ¿Cómo FastAPI sabe que es request body?

Porque el parámetro es un modelo Pydantic.

◆ Flujo interno

1. Cliente envía JSON
2. Unicorn recibe request
3. FastAPI enruta
4. Pydantic valida
5. Si todo OK → ejecuta función
6. Se serializa respuesta a JSON
7. Devuelve 200

Si falta un campo obligatorio:

→ 422 Unprocessable Entity

5 Response Body

El **response body** es el payload que el servidor devuelve.

En FastAPI:

- Se convierte automáticamente a JSON
- Se incluye Content-Type: application/json

6 model_dump() (Antes dict())

En Pydantic v2 se utiliza:

```
item_dict = item.model_dump()
```

Convierte el modelo a diccionario Python.

◆ Ejemplo con cálculo adicional

```
@app.post("/items/")  
async def create_item(item: Item):  
    item_dict = item.model_dump()
```

```
if item.tax:
    price_with_tax = item.price + item.tax
    item_dict.update({"price_with_tax": price_with_tax})

return item_dict
```

7 Desempaquetado de Diccionarios

El operador:

****diccionario**

Inserta claves y valores dentro de otro diccionario.

Ejemplo:

```
return {"item_id": item_id, **item.model_dump()}
```

Resultado:

```
{
  "item_id": 10,
  "name": "Laptop",
  "price": 1000
}
```

8 Combinación: Body + Path + Query

```
@app.put("/items/{item_id}")
```

```
async def update_item(
```

```
    item_id: int,
```

```
    item: Item,
```

```
    q: str | None = None
```

```
):
```

```
    item_dict = item.model_dump()
```

```
    return {
```

```
        "item_id": item_id,
```

```
        **item_dict,
```

```
        "q": q
```

```
    }
```

FastAPI distingue automáticamente:

- Si está en la ruta → Path
- Si es modelo Pydantic → Body
- Si es tipo simple y no está en la ruta → Query

9 Validación Avanzada con Annotated

Python permite metadata en anotaciones usando:

```
from typing import Annotated
```

FastAPI usa esa metadata para validar.

◆ Validación de Query (Strings)

```
from fastapi import Query
from typing import Annotated
```

```
@app.get("/items/")
async def read_items(
    q: Annotated[str | None, Query(min_length=3, max_length=50)] = None
):
    return {"q": q}
```

Si no cumple:

→ 422

10 Validación Numérica en Path

```
from fastapi import Path
```

```
@app.get("/items/{item_id}")
async def read_item(
    item_id: Annotated[int, Path(gt=0, le=1000)],
    q: str
):
    return {"item_id": item_id}
```

Reglas:

- gt → greater than
- ge → greater or equal
- lt → less than
- le → less or equal

1 1 Modelo de Parámetros Query

Cuando hay muchos filtros:

```
from pydantic import BaseModel, Field
from typing import Literal
```

```
class FilterParams(BaseModel):
    limit: int = Field(10, ge=1, le=100)
    offset: int = Field(0, ge=0)
    order_by: Literal["created_at", "updated_at"] = "created_at"
    tags: list[str] = []
```

Uso:

```
from fastapi import Query
from typing import Annotated
```

```
@app.get("/items/")
async def read_items(
    filters: Annotated[FilterParams, Query()]
):
    return filters
```

Ventajas:

- Centraliza validación
- Reutilizable
- Código más limpio
- Contrato más claro

1 2 ¿Por Qué Validar en la Entrada?

La validación en la entrada es:

- ✓ Primera línea de defensa
- ✓ Mejora seguridad
- ✓ Evita estados inválidos
- ✓ Reduce código defensivo
- ✓ Refuerza contrato de API
- ✓ Mejora documentación automática

La validación ocurre antes de ejecutar lógica de negocio.

Resumen General

- ✓ Request body viaja en JSON
- ✓ Pydantic valida automáticamente
- ✓ FastAPI responde 422 ante error
- ✓ `model_dump()` convierte a diccionario
- ✓ Se pueden combinar Body + Path + Query
- ✓ Annotated permite agregar metadata
- ✓ Query y Path permiten reglas declarativas
- ✓ Modelos Query agrupan filtros complejos